

TraceSQL: Traceable Answerability Estimation for Reference-Free Text-to-SQL Verification

Neelesh Kumar Shukla*
Oracle Corporation
India

Debasmita Panda*
Oracle Corporation
India

Srutanik Bhaduri
Oracle Corporation
India

Aditya Banerjee
Oracle Corporation
United States

Viji Krishnamurthy
Oracle Corporation
United States

Abstract

Text-to-SQL systems are commonly evaluated using ground-truth SQL queries or reference execution results, but such supervision is unavailable at inference time in real-world deployments. This creates a critical verification problem: given only a user question, database context, and generated SQL, can a system estimate whether the generated query is likely to correctly answer the question? Recent approaches use LLMs as judge or specialized agents to inspect generated SQL, but their decisions can be difficult to trace. Outcome Reward Models (ORMs) address this by learning from execution-labeled candidate SQLs and assigning correctness scores to unseen queries, yet they still provide limited visibility into the signals behind each verification. To address this limitation, we propose **TraceSQL**, a lightweight and traceable verification model built on explicit diagnostic features. TraceSQL combines 67 features capturing question ambiguity, question requirements, question–schema–SQL consistency, SQL structure, and intent alignment. These signals remain available for examining which factors influence each prediction and for tracing decisions back to diagnostic evidence. On BIRD development databases, TraceSQL achieves 66.47% F1 and 64.48% ROC-AUC, compared with 61.87% F1 and 58.26% ROC-AUC for the GradeSQL-7B ORM baseline on the same generated-SQL evaluation. Feature attribution further shows that the model relies on both semantic grounding and deterministic SQL-structure signals. These results show that SQL verification can be performed with a lightweight learned model while retaining feature-level evidence for inspecting and diagnosing its predictions.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Machine learning*; • **Information systems** → *Database query processing*.

Keywords

Text-to-SQL, answerability estimation, verification, interpretability, traceability, AutoML, BIRD

1 Introduction

Text-to-SQL systems have advanced rapidly with large language models (LLMs). Benchmarks such as Spider [15] and BIRD [8] evaluate systems on complex questions and unseen database schemas. Recent methods further improve generation through decomposition,

example selection, self-correction, multi-agent reasoning, and large-scale synthetic supervision [3, 7, 10, 13]. Despite these advances, generated SQL can contain subtle semantic, schema-grounding, or structural errors. Reliable Text-to-SQL therefore requires not only generating a query, but also determining whether the generated query can be trusted. During benchmark evaluation, this decision can be supported by ground-truth SQL or reference execution results. In deployment, however, such references are typically unavailable, so post-generation verification must rely on the user question, database context, generated SQL, and signals derived from their relationships. Existing approaches provide post-generation reliability in different ways. LLM-based methods use self-correction or specialized agents to inspect and refine generated SQL [1, 10, 13]. More recent work has also explored reasoning-based SQL judges and explicit rule-based verification [2, 11]. These approaches showcase different forms of verification evidence, such as correction reasoning, agent feedback, execution signals, and rule-based checks. However, this evidence is usually tied to the design of the individual method and is not represented in a unified, measurable form that can be systematically analyzed or linked to a learned verification decision.

Outcome Reward Models (ORMs) provide a learned alternative to these approaches. GradeSQL [12] generates multiple candidate SQLs during training, derives correctness labels through execution match with the reference SQL, and fine-tunes an LLM to estimate candidate correctness. At inference time, the trained ORM assigns a probability-based correctness score without requiring the reference SQL. This demonstrates that execution-derived supervision can be used to train a dedicated Text-to-SQL verifier. However, the resulting prediction is primarily exposed as a scalar score. While useful for ranking or selecting candidates, this score provides limited visibility into the specific semantic or structural signals that contributed to the decision.

This leaves an important gap when verification is expected to support not only prediction, but also understanding and diagnosis. A candidate may fail because of incorrect schema grounding, unsupported analytical requirements, missing or inappropriate SQL operations, incorrect grouping, or a mismatch with the intended result. Knowing that a candidate received a low verification score does not by itself distinguish among these cases. For a verifier to support system analysis, its decision should be *interpretable* in terms of meaningful signals, *explainable* in terms of how those signals influence the prediction, and *traceable* back to the evidence from which those signals were derived.

*Equal contribution.

We introduce **TraceSQL**, a lightweight learned verifier designed around these three properties. TraceSQL represents each question-candidate pair through explicit diagnostic signals rather than relying only on a final verifier score. The representation combines five complementary diagnostic families: question ambiguity, question planning, pair analysis between the question, schema, and generated SQL, deterministic SQL structure, and intent alignment. Together, they produce 67 compact features describing the requirements expressed by the question, the grounding and semantic consistency of the candidate, the operations present in the SQL, and its alignment with the intended request.

The representation provides three levels of visibility into the verification process. First, the model inputs are named diagnostic features with explicit semantic meaning, making the representation itself interpretable. Second, feature-importance methods can be applied to the learned model to determine which signals influence its predictions, providing model-level explainability. Third, the richer diagnostic outputs from which the features are derived are retained separately, allowing influential features to be traced back to their supporting evidence. TraceSQL therefore connects the final verification decision to both measurable feature-level signals and their diagnostic provenance.

We train TraceSQL using generated candidate SQLs from the balanced BIRD ORM training data released with GradeSQL. We then evaluate TraceSQL and GradeSQL-7B on the same generated-candidate SQLs from held-out BIRD databases. TraceSQL achieves 66.47% F1 and 64.48% ROC-AUC, compared with 61.87% F1 and 58.26% ROC-AUC for GradeSQL-7B. Beyond predictive performance, our feature analysis shows that the learned verifier draws on both semantic diagnostic signals and deterministic SQL-structure properties, allowing its behavior to be examined in terms of concrete properties of the generated query.

Contributions. We make three main contributions. First, we introduce **TraceSQL**, a lightweight learned verifier that combines SQL verification with interpretability, explainability, and traceability. Second, we develop a 67-feature representation from five diagnostic families in which model inputs remain semantically meaningful and retain provenance to their underlying diagnostic evidence. Third, we evaluate TraceSQL against GradeSQL-7B on identical held-out generated-candidate SQLs and analyze the semantic and structural signals that drive its predictions, showing how verification decisions can be connected to explicit feature-level evidence.

2 Related Work

2.1 Text-to-SQL Generation

Recent LLM-based Text-to-SQL methods improve different parts of the generation process. DIN-SQL decomposes generation into schema linking, query decomposition, SQL generation, and self-correction [10]. DAIL-SQL focuses on prompt construction and example selection for in-context learning [3]. MAC-SQL uses a multi-agent framework with dedicated components for decomposition, context reduction, and SQL refinement [13]. OmniSQL addresses the training-data bottleneck through large-scale synthetic supervision for specialized Text-to-SQL models [7]. These methods mainly

improve SQL generation or refinement, whereas TraceSQL operates after generation and focuses on verifying a produced candidate while preserving the evidence behind the verification decision.

2.2 Post-Generation Verification and Learned Verifiers

Recent work has increasingly focused on detecting and correcting errors after SQL generation. MAGIC uses specialized agents to derive self-correction guidelines from generation failures [1], while DPC performs training-free candidate verification by comparing SQL behavior with an independently constructed execution path [6]. Learned verification has also emerged as an alternative: STaR-SQL incorporates an outcome-supervised reward model to rank generated candidates during test-time reasoning [5].

GradeSQL [12] develops this direction further through task-specific Outcome Reward Models (ORMs), trained on execution-derived candidate labels and used to assign continuous correctness scores at inference time. TraceSQL follows the idea of learning from execution-derived candidate labels, but differs in how verification is represented. Rather than exposing only a scalar correctness score, TraceSQL retains a fixed set of diagnostic signals that can be analyzed individually and traced back to their underlying evidence.

2.3 Interpretable, Explainable, and Traceable Verification

Recent verification methods provide different forms of transparency, including correction guidelines, structured reasoning, and explicit verification constraints [1, 2, 11]. TraceSQL takes a complementary approach by representing verification evidence as explicit, measurable features that can be directly related to the behavior of the learned verifier.

TraceSQL provides transparency at three levels. *Interpretability* comes from semantically defined diagnostic features used as model inputs. *Explainability* is provided through feature-importance and attribution methods, including permutation importance and SHAP [9], which reveal which signals influence the model’s predictions. *Traceability* is preserved by linking these features back to the diagnostic evidence from which they were derived. Together, these properties allow the verifier to expose not only its prediction, but also the semantic and structural signals associated with that decision.

3 Task Formulation

Let q denote a natural-language question, s the corresponding database context, and x a candidate SQL to be verified. The verification task is to estimate whether x correctly answers q using only information available to the verifier. In the primary setting, x is a generated candidate SQL; the same verification pipeline is also applied to the ground-truth SQL provided with the BIRD development databases.

During training and evaluation, candidate correctness is determined using execution match:

$$y = \mathbb{1}[\text{Exec}(x) = \text{Exec}(x^*)], \quad (1)$$

where x^* denotes the reference SQL and $y \in \{0, 1\}$ is the candidate-level correctness label. The reference SQL and the resulting execution-match label are used only to construct offline supervision and are not provided as inputs to TraceSQL.

Given (q, s, x) , the diagnostic pipeline constructs a 67-dimensional feature representation

$$\mathbf{z} = f(q, s, x), \quad (2)$$

where $\mathbf{z}$ captures explicit signals related to question ambiguity, question requirements, question–schema–SQL consistency, SQL structure, and intent alignment. TraceSQL then estimates

$$\hat{p} = P(y = 1 \mid \mathbf{z}), \quad (3)$$

where $\hat{p}$ is the estimated probability that the candidate SQL is correct.

Using a fixed decision threshold $\tau = 0.50$, the final verification decision is

$$\hat{y} = \mathbb{I}[\hat{p} \geq \tau]. \quad (4)$$

The features in $\mathbf{z}$ remain individually inspectable and retain their diagnostic provenance. This allows the verification decision to be examined in terms of the semantic, grounding, and structural signals that influence the learned prediction.

4 Traceable Answerability Signals

4.1 Diagnostic Evidence Generation

TraceSQL derives its verification evidence from three upstream diagnostic modules: the Ambiguity Detector, Pair Analyzer, and SQL Repair Module. Together, they capture question-level ambiguity, question–schema–SQL consistency, and alignment between the user request and the candidate SQL. Their structured outputs are retained as diagnostic evidence and subsequently transformed into the model features described in Section 4.2.

4.1.1 Ambiguity Detector. The Ambiguity Detector assesses whether the user question is sufficiently specified with respect to the available database context. It produces:

- **Ambiguity status:** indicates whether the question is considered ambiguous.
- **Ambiguity probability:** estimates the degree of ambiguity on a 0–100 scale.
- **Explanation:** provides the evidence or rationale supporting the ambiguity assessment.

These outputs provide the source evidence for constructing the downstream ambiguity features.

4.1.2 Pair Analyzer. The Pair Analyzer evaluates whether the candidate SQL is supported by the user question and database context. It applies predefined verification rules covering schema and column grounding, joins, identifiers, data types, temporal semantics, metrics, null and validity logic, and business-term mappings. Its diagnostic outputs include:

- **Support status:** classifies the candidate as *supported*, *partially supported*, *unsupported*, or *unclear*.
- **Pair score and summary:** provide an overall assessment of candidate support.
- **Rule-level results:** record the status, score, explanation, and supporting evidence for each verification rule.

- **Findings:** describe detected grounding or consistency gaps and the affected database or SQL elements.

The rule-level statuses are used to construct the Pair Analysis feature family, while the richer explanations and findings are retained as diagnostic evidence for traceability.

4.1.3 SQL Repair Module. The SQL Repair Module is used in diagnosis-only mode to assess how well the candidate SQL reflects the requirements expressed by the user question. It decomposes the question into structured intents, explains the behavior of the candidate SQL, and evaluates their alignment. The outputs retained for this work are:

- **Question intent breakdown:** decomposes the question into ordered, schema-grounded requirements that the SQL is expected to satisfy.
- **SQL explanation:** describes the operations performed by the candidate SQL, including tables, columns, joins, filters, aggregations, and other query constructs.
- **Reference-free evaluation:** assesses the candidate against the planned question intents and produces an evaluation score, confidence, gate decision, intent-level alignment assessments, and an intervention reason when a mismatch is detected.

These outputs provide the source evidence for constructing the downstream question-planning and intent-alignment features.

4.2 Feature Extraction

Feature extraction depends on the type of diagnostic evidence. For the ambiguity, question-planning, and intent-alignment families, TraceSQL uses predefined probe sets encoded in fixed evaluation prompts. Relevant diagnostic units—including ambiguity claims, question-intent items, and intent-alignment evidence—are supplied to an LLM together with the required question, schema, and SQL context. The LLM evaluates each unit against the applicable probes and returns a structured verdict (*PASS*, *FAIL*, *N/A*, or *UNKNOWN*). The LLM configurations used across the diagnostic pipeline are reported in Section 7.1.

Pair Analysis requires no additional LLM assessment during feature construction; the statuses of predefined canonical rules are projected directly into the model representation. SQL-structure features are also deterministic: the candidate SQL is parsed into an abstract syntax tree (AST) using SQLGlot, from which predefined structural properties are extracted. Figure 1 summarizes the resulting feature-construction pipeline.

4.2.1 Ambiguity Features. The ambiguity feature pipeline transforms the Ambiguity Detector output into ten structured features. Ambiguity status and probability are retained directly, while the detector explanation is decomposed into claims and evaluated against eight predefined ambiguity probes covering metric, entity, temporal, constraint, interpretation, reference, external-context, and analytical-intent ambiguity. Claim-level assessments are aggregated into one verdict per probe, yielding eight probe features in addition to the two detector-level signals.

4.2.2 Question-Planning Features. The question-planning feature pipeline transforms the *question intent breakdown* produced by the

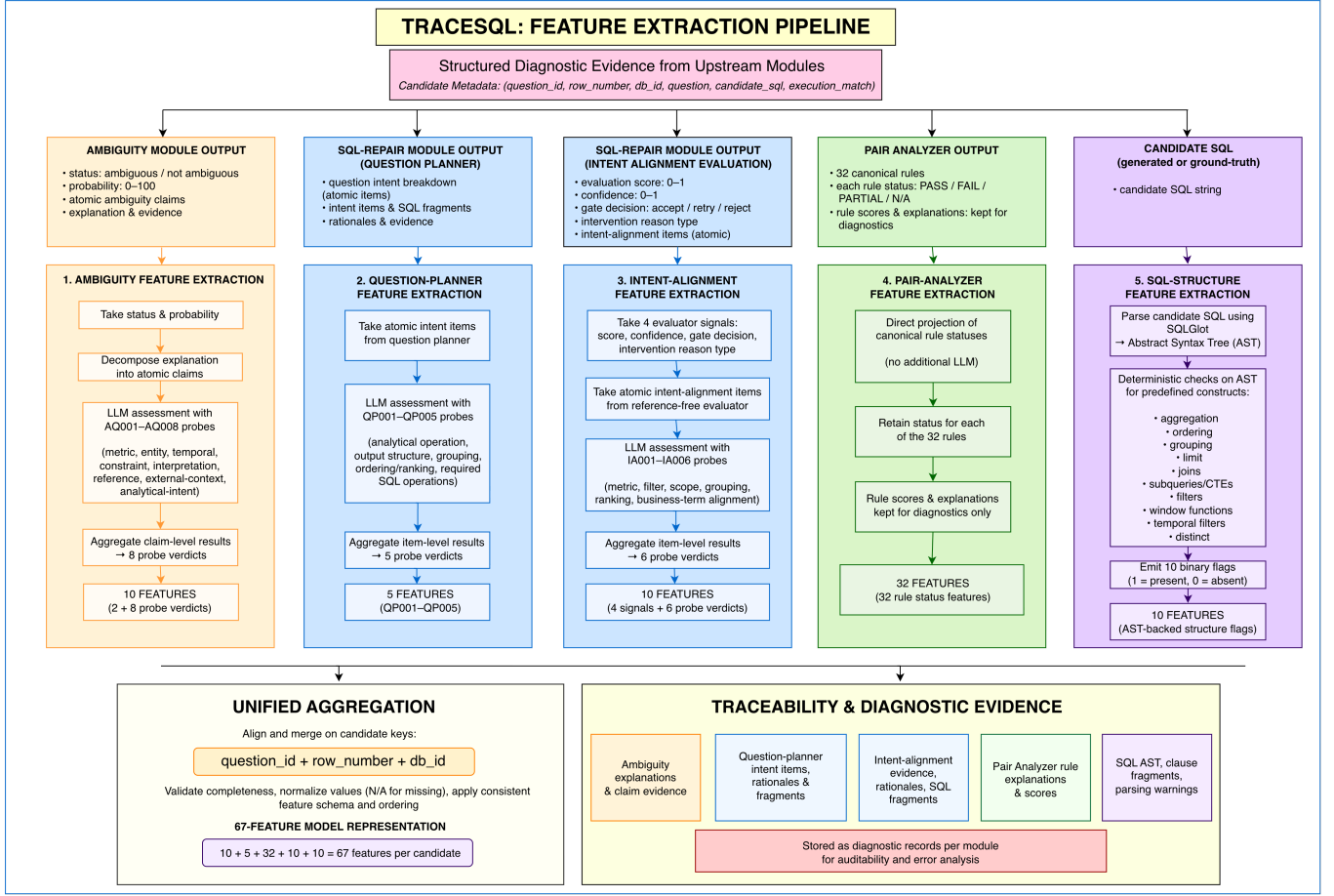


Figure 1: TraceSQL feature-construction pipeline. Diagnostic evidence is transformed into five feature families through LLM-based probe assessment, direct projection of Pair Analyzer rule statuses, and deterministic SQL-AST extraction using SQLGlot. The resulting $10 + 5 + 32 + 10 + 10 = 67$ features are aligned at the candidate level and merged into the unified model representation, while richer diagnostic evidence is retained separately for traceability.

SQL Repair Module into five structured features. Each atomic intent item is evaluated by an LLM against five predefined planning probes covering the analytical operation, expected output structure, grouping, ordering or ranking, and required SQL operations. Item-level assessments are aggregated into one verdict per probe, producing five candidate-level question-planning features. Supporting evidence, rationale, confidence, and planning fragments are retained separately for traceability.

4.2.3 Pair-Analysis Features. The pair-analysis feature pipeline directly transforms the rule-level outputs of the Pair Analyzer into 32 structured features. Unlike the ambiguity and question-planning pipelines, no additional LLM assessment is performed during this stage. Each predefined Pair Analyzer rule contributes one categorical feature corresponding to its rule status, while the associated rule score and explanation are retained separately as diagnostic evidence. The rules cover schema and scope grounding, joins, identifiers, data types, temporal semantics, metric definitions, data validity, and business-term mappings.

4.2.4 SQL-Structure Features. The SQL-structure pipeline extracts ten deterministic features directly from the candidate SQL. Each query is parsed into an abstract syntax tree (AST) using SQLGlot, which is inspected for aggregation, grouping, joins, filtering, temporal filtering, ordering, limits, subqueries or CTEs, window functions, and DISTINCT. Each property is represented as a binary feature indicating its presence or absence. The parsed AST, clause-level SQL evidence, and parsing warnings are retained separately for traceability.

4.2.5 Intent-Alignment Features. The intent-alignment feature pipeline transforms the structured output of the reference-free evaluator into ten candidate-level features. Four evaluator signals—the overall evaluation score, confidence, gate decision, and intervention-reason type—are retained directly. In addition, each intent-alignment item is evaluated by an LLM against six predefined probes covering metric, filter, scope, grouping, ranking, and business-term alignment. Item-level assessments are aggregated into one verdict per

probe, producing six additional features. Supporting evidence, rationales, SQL fragments, and the original intent-alignment records are retained separately for traceability.

Table 1 summarizes the resulting 67-dimensional diagnostic representation and provides representative signals from each feature family. Complete definitions of all 67 features, including their probe and canonical-rule identifiers, are provided in the supplementary material.

5 Research Questions

We evaluate TraceSQL from four complementary perspectives: verification performance, diagnostic-signal importance, cross-database generalization, and traceability. We investigate the following research questions.

- RQ1:* How effectively can structured diagnostic evidence support reference-free Text-to-SQL verification?
- RQ2:* Which diagnostic signals contribute most to the learned verification decision?
- RQ3:* How well does TraceSQL generalize across held-out databases?
- RQ4:* To what extent can TraceSQL predictions be traced back to explicit diagnostic evidence?

6 Data Preparation

We use the balanced BIRD ORM training corpus released with GradeSQL [12]. After attaching database identifiers, the corpus contains 30,686 candidate SQL queries from 69 databases and 2,642 unique question-database groups. Each group contains between 2 and 32 candidates, so sampling individual rows would disproportionately represent questions with larger candidate sets.

Database identifiers are assigned by matching normalized question-plus-evidence text to the corresponding BIRD training records. We then remove only exact duplicates across question, schema, SQL, data, label, and database ID. This reduces the corpus from 30,686 to 19,592 candidates while preserving alternative candidate SQLs for the same question.

For each eligible question-database group, we retain one positive and one negative candidate, yielding 2,642 balanced pairs, or 5,284 candidates. From these, we select 1,000 complete pairs (2,000 candidates) using database-stratified sampling. All 69 training databases remain represented, with the remaining samples allocated proportionally across databases. The resulting training set contains 1,000 positive and 1,000 negative candidates and is reproducible using seed 42.

The final 2,000-candidate sample represents approximately 6.5% of the original ORM corpus and 37.9% of the 5,284-candidate paired source. We use this sample for the current TraceSQL training experiments, while scaling to the complete paired source is left for future work.

7 Experimental Setup

7.1 LLM Configuration for Diagnostic Evidence and Feature Extraction

The diagnostic pipeline uses fixed LLM backends for evidence generation and LLM-based feature extraction. The Ambiguity Detector and SQL Repair Module use GPT-4o, while the Pair Analyzer uses

GPT-5. 2. In the unified feature-extraction pipeline, the ambiguity, question-planning, and intent-alignment probe assessments are performed using GPT-4o.

7.2 Training with FLAML

We train TraceSQL using FLAML [14] with all 67 diagnostic features as input. The training set contains 2,000 candidates, balanced between 1,000 positive and 1,000 negative `execution_match` labels. Following FLAML’s preprocessing and feature transformation, 62 features are retained as inputs to the selected estimator. Thus, TraceSQL is defined over the complete 67-feature diagnostic representation, while the fitted estimator operates on the corresponding 62-feature transformed representation.

FLAML uses internal validation ROC-AUC for model selection and subsequently refits the selected estimator on all 2,000 training candidates. Metrics computed on these candidates therefore describe training-population performance rather than held-out evaluation.

With a 1,800-second search budget, FLAML evaluates 4,228 trials and selects an Extra Trees classifier [4] with seven trees, entropy-based splitting, `max_leaves=6`, and `max_features=0.2241`. Increasing the search budget to 3,600 seconds returns the same model configuration and performance, indicating that the search had stabilized within the shorter budget.

7.3 Baseline and External Evaluation

The main baseline is GradeSQL-7B [12]. For external evaluation, the saved 1,800-second TraceSQL model is applied without refitting to 11 BIRD development databases that are disjoint from the training databases. We evaluate two complementary settings. The generated-SQL setting contains both execution-matching and non-matching candidates and serves as the primary two-class verification evaluation. The ground-truth SQL setting evaluates the ground-truth SQL provided with the BIRD development databases and serves as a complementary assessment of verifier behavior on these queries.

GradeSQL-7B and TraceSQL are evaluated on identical generated-candidate SQLs with the same execution-match labels. This provides a matched evaluation of the two verification approaches on the same test instances.

8 Results

8.1 Training Results

The selected Extra Trees model achieves an internal validation ROC-AUC of 0.6109. After refitting on all 2,000 training candidates, it reaches an accuracy of 0.6085, precision of 0.5987, recall of 0.6580, F1 of 0.6270, and ROC-AUC of 0.6520. These values characterize model development and training-population performance rather than held-out evaluation.

8.2 Which Diagnostic Signals Does the Model Use?

We analyze the fitted 1,800-second model using native Extra Trees feature importance, permutation importance, and exact Tree SHAP [9]. We use these methods to characterize which diagnostic signals the learned verifier relies on. Permutation importance and SHAP

Table 1: Overview of the five TraceSQL diagnostic feature families and representative signals captured by each family.

Feature Family	#	Representative Features	Captured Evidence
Ambiguity	10	Ambiguity Overall Status; Metric Clarity; Time-Scope Clarity	Whether the question is sufficiently specified with respect to the requested metric, entities, temporal scope, constraints, references, and analytical intent.
Question Planning	5	Question Planner: Analytical Operation; Question Planner: Output Structure; Question Planner: Grouping Join-Key Support;	Whether the candidate SQL reflects the analytical operation, expected output structure, grouping, ordering, and SQL operations required by the question.
Pair Analysis	32	Data-Validity Rule Support; Additional Grounding Check SQL Aggregation Usage;	Whether tables, columns, joins, identifiers, data types, temporal semantics, metrics, validity rules, and business concepts are grounded in the available database context.
SQL Structure	10	SQL LIMIT Usage; SQL DISTINCT Usage Intent Alignment: Evaluation Score;	Deterministic structural properties of the candidate SQL extracted from its SQLGlot abstract syntax tree.
Intent Alignment	10	Intent Alignment: Grouping; Intent Alignment: Business Term	Whether the candidate SQL aligns with the requested metric, filters, scope, grouping, ranking, and business terminology.
Total	67		

provide complementary perturbation-based and attribution-based views, while native Extra Trees importance is used as a supporting analysis.

The leading signals span both deterministic SQL structure and semantic diagnostic evidence. Permutation importance ranks *SQL DISTINCT Usage*, *SQL LIMIT Usage*, *SQL Aggregation Usage*, *Additional Grounding Check*, *Pattern-Based Business Meaning Support*, and *Data-Validity Rule Support* among the strongest features. SHAP independently highlights the same broad set. Nine features appear within the top ten under both permutation importance and SHAP: *Additional Grounding Check*, *SQL LIMIT Usage*, *Pattern-Based Business Meaning Support*, *SQL DISTINCT Usage*, *SQL Aggregation Usage*, *Data-Validity Rule Support*, *SQL Grouping Usage*, *Intent Alignment: Grouping*, and *Question Planner: Output Structure*. This agreement shows that the model relies on a combination of deterministic SQL-structure signals and semantic diagnostic evidence rather than on a single type of signal.

Table 2 reports the six highest-ranked features by permutation importance together with their mean absolute SHAP values.

The strongest signals are distributed across multiple parts of the TraceSQL representation. SQL-structure features such as *SQL DISTINCT Usage*, *SQL LIMIT Usage*, and *SQL Aggregation Usage* appear alongside grounding and semantic-validity signals from the Pair Analysis family. The fitted verifier therefore combines evidence about the operations present in the candidate SQL with evidence about whether those operations are supported by the question and database context.

A more detailed feature-level analysis, including global SHAP analysis, cross-method comparison of SHAP, permutation importance, and native Extra Trees importance, feature correlation analysis, and a representative prediction trace, is provided in the supplementary material.

Table 2: Leading features by permutation importance in the selected TraceSQL model. Permutation values are ROC-AUC decreases after shuffling; SHAP values are mean absolute contributions.

Feature	Perm.	Mean SHAP
SQL DISTINCT Usage	0.0344	0.0074
SQL LIMIT Usage	0.0335	0.0147
SQL Aggregation Usage	0.0327	0.0073
Additional Grounding Check	0.0317	0.0199
Pattern-Based Business Meaning Support	0.0219	0.0117
Data-Validity Rule Support	0.0165	0.0070

Table 3: Overall generated-SQL performance on 1,521 candidates from 11 held-out BIRD development databases. Bold indicates the better result for each metric.

Model	Acc.	Prec.	Rec.	F1	AUC
GradeSQL-7B	57.46	63.64	60.21	61.87	58.26
TraceSQL	62.46	68.11	64.91	66.47	64.48

8.3 Generated-SQL Verification and Cross-Database Generalization

We evaluate TraceSQL and GradeSQL-7B on 1,521 generated SQLs from 11 held-out BIRD development databases. As shown in Table 3, TraceSQL outperforms GradeSQL-7B across all aggregate metrics, with improvements of 4.60 percentage points in F1 and 6.22 points in ROC-AUC. These results indicate that the structured diagnostic representation supports effective verification on databases not observed during training.

Table 4 shows that TraceSQL’s aggregate gains are reflected across a broad range of held-out databases. TraceSQL achieves stronger results on most reported metrics for nine of the eleven databases, including *California Schools*, *Codebase Community*, *Financial*, *Superhero*, and *Toxicology*. Particularly large improvements are

observed for databases such as *Superhero* and *Thrombosis Prediction*. Performance is more competitive on *Debit Card Specializing*, while *Formula 1* is the main case where GradeSQL-7B remains stronger on most metrics. Overall, the database-level results suggest that the TraceSQL representation transfers effectively across diverse schemas and query distributions, while leaving room for further improvement on specific domains.

8.4 Traceability of Verification Decisions

TraceSQL preserves the connection between influential model signals and explicit diagnostic evidence through the feature-extraction process: every model feature is associated with a named probe, canonical verification rule, or deterministic SQL-structure property, while the richer supporting evidence is retained under the same candidate identity.

For example, SQL-structure features such as *SQL LIMIT Usage* can be traced to the corresponding AST node and SQL clause; Pair Analyzer features such as *Additional Grounding Check* and *Pattern-Based Business Meaning Support* map to canonical rule statuses and their supporting explanations; and features such as *Intent Alignment: Grouping* and *Question Planner: Output Structure* map to probe verdicts, supporting evidence, rationales, and associated diagnostic records.

This provides a direct provenance path from an influential model feature to the diagnostic condition from which it was derived. A representative candidate-level prediction trace illustrating this feature-to-evidence connection for a false-positive verification decision is provided in the supplementary material.

8.5 Verification on Ground-Truth SQL

As a complementary evaluation, we apply both verifiers to 1,534 ground-truth SQL queries from the BIRD development databases. Unlike the generated-SQL evaluation, this setting contains only the reference SQL associated with each question and therefore contains no negative candidates. It measures how often each verifier assigns a positive decision when evaluated on the available ground-truth SQL. Since all instances belong to the positive class, accuracy is equivalent to recall, precision is 100% by construction, and ROC-AUC is undefined.

As shown in Table 5, GradeSQL-7B accepts 62.84% of the ground-truth SQL queries, compared with 60.17% for TraceSQL. At the database level, TraceSQL achieves higher acceptance on California Schools, Codebase Community, Financial, Formula 1, and Thrombosis Prediction, while GradeSQL-7B is higher on the remaining databases. Complete database-level results are provided in the supplementary material.

We use this experiment as a complementary evaluation of verifier behavior on ground-truth SQL, while the generated-SQL setting remains the primary evaluation for comparing binary verification performance.

9 Discussion

9.1 Answerability Through Diagnostic Verification Evidence

TraceSQL and GradeSQL use the same type of execution-derived candidate supervision, but TraceSQL exposes a structured diagnostic representation at inference time rather than only a final correctness score. This representation combines question-level signals, such as ambiguity and planning requirements, with candidate-level signals covering schema grounding, SQL structure, and intent alignment.

This distinction is useful because verification depends on both the question and the candidate SQL. A clearly specified question can still be paired with a candidate that does not realize the intended request, while ambiguity in the question can make the reliability of an otherwise valid candidate SQL harder to assess. TraceSQL captures these complementary signals explicitly and uses them to support reference-free verification.

9.2 Traceability and Generality

The feature analysis shows that the learned verifier relies on multiple forms of evidence. Deterministic SQL-structure features such as *SQL DISTINCT Usage*, *SQL LIMIT Usage*, and *SQL Aggregation Usage* appear alongside grounding and semantic-validity signals from the Pair Analysis family, such as *Additional Grounding Check*, *Pattern-Based Business Meaning Support*, and *Data-Validity Rule Support*. This suggests that the model combines information about what operations are present in the SQL with evidence about whether those operations are supported by the question and database context.

Because the features have fixed semantic definitions and remain linked to their underlying diagnostic records, influential predictions can be inspected and traced back to concrete evidence. The full diagnostic vocabulary is retained in the representation, allowing signals that are weak on the current BIRD distribution to remain available under different database and query settings.

10 Future Work

A primary direction for future work is to scale TraceSQL training from the current 2,000-candidate controlled sample to the complete 5,284-candidate paired source while retaining the same held-out 11-database evaluation. This will allow us to examine whether additional supervision improves verification performance and whether the learned importance of the diagnostic signals remains stable as the training set expands.

We also plan to investigate a tighter integration between TraceSQL and GradeSQL-style outcome reward modeling. Rather than using the diagnostic representation as a separate verifier, the 67 TraceSQL features can be incorporated directly into the GradeSQL verification model as additional structured evidence. This would enable a controlled study of how augmenting an ORM with explicit ambiguity, planning, grounding, SQL-structure, and intent-alignment signals affects predictive performance relative to the original GradeSQL formulation.

This comparison will help clarify whether the TraceSQL representation is more useful as a standalone verifier or as an additional evidence layer within a larger learned verification model.

Table 4: Database-level generated-SQL performance. Bold indicates the better result for each metric within a database.

Database	GradeSQL-7B					TraceSQL				
	Acc.	Prec.	Rec.	F1	AUC	Acc.	Prec.	Rec.	F1	AUC
California Schools	45.45	64.86	40.68	50.00	58.33	51.14	69.05	49.15	57.43	56.25
Card Games	55.26	50.47	62.79	55.96	53.98	61.05	56.82	58.14	57.47	64.73
Codebase Community	61.83	73.45	66.94	70.04	59.31	65.59	75.00	72.58	73.77	66.03
Debit Card Specializing	48.33	44.19	73.08	55.07	46.27	60.00	55.00	42.31	47.83	60.35
European Football 2	57.14	71.43	55.56	62.50	59.48	61.90	75.38	60.49	67.12	69.36
Financial	57.55	56.92	68.52	62.18	56.91	64.15	63.79	68.52	66.07	63.46
Formula 1	61.27	65.98	65.31	65.64	63.31	60.12	65.93	61.22	63.49	65.27
Student Club	60.65	70.64	72.64	71.63	47.19	61.94	71.56	73.58	72.56	57.20
Superhero	54.26	73.49	62.24	67.40	36.87	68.99	81.52	76.53	78.95	62.66
Thrombosis Prediction	57.67	45.45	30.77	36.70	56.89	61.96	52.11	56.92	54.41	60.76
Toxicology	60.69	64.06	54.67	58.99	56.91	65.52	66.67	66.67	66.67	67.61

Table 5: Verification performance on 1,534 ground-truth SQL queries from the BIRD development databases. Since all instances belong to the positive class, Accuracy equals Recall, Precision is 100%, and ROC-AUC is undefined.

Model	Acc./Rec.	Prec.	F1	AUC
GradeSQL-7B	62.84	100.00	77.18	N/A
TraceSQL	60.17	100.00	75.13	N/A

11 Conclusion

We presented TraceSQL, a reference-free Text-to-SQL verifier built around traceable diagnostic evidence. TraceSQL constructs a 67-feature diagnostic representation covering ambiguity, question requirements, question-schema-SQL consistency, SQL structure, and intent alignment, while retaining the underlying diagnostic evidence for further inspection.

On held-out BIRD development databases, TraceSQL improves over GradeSQL-7B, reaching 66.47% F1 and 64.48% ROC-AUC compared with 61.87% and 58.26%, respectively. The feature analysis also shows that the verifier draws on both deterministic SQL-structure signals and semantic grounding evidence, suggesting that reliable verification benefits from combining multiple forms of diagnostic information.

A key property of TraceSQL is that the verification process remains inspectable. Influential features can be connected to meaningful diagnostic conditions and traced back to the evidence from which they were derived. The results show that reference-free Text-to-SQL verification can be learned from structured diagnostic signals while preserving evidence that supports inspection and failure analysis.

References

- [1] Arian Askari, Christian Poelitz, and Xinye Tang. 2025. MAGIC: Generating Self-Correction Guideline for In-Context Text-to-SQL. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 23433–23441. doi:10.1609/aaai.v39i22.34511
- [2] Jiayuan Bai, Xuan guang Pan, Chongyang Tao, and Shuai Ma. 2025. JudgeSQL: Reasoning over SQL Candidates with Weighted Consensus Tournament. *arXiv preprint arXiv:2510.15560* (2025).
- [3] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *arXiv preprint arXiv:2308.15363* (2023).
- [4] Pierre Geurts, Damien Ernst, and Louis Wehenkel. 2006. Extremely Randomized Trees. *Machine Learning* 63, 1 (2006), 3–42. doi:10.1007/s10994-006-6226-1
- [5] Mingqian He, Yongliang Shen, Wenqi Zhang, Qiuying Peng, Jun Wang, and Weiming Lu. 2025. STaR-SQL: Self-Taught Reasoner for Text-to-SQL. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 24365–24375. doi:10.18653/v1/2025.acl-long.1187
- [6] Boyan Li, Ou Ocean Kun Hei, Yue Yu, and Yuyu Luo. 2026. DPC: Training-Free Text-to-SQL Candidate Selection via Dual-Paradigm Consistency. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 6897–6913. doi:10.18653/v1/2026.acl-long.313
- [7] Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, Hong Chen, and Cuiping Li. 2025. OmniSQL: Synthesizing High-Quality Text-to-SQL Data at Scale. *arXiv preprint arXiv:2503.02240* (2025).
- [8] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chen-Chuan Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [9] Scott M. Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*, Vol. 30. 4765–4774.
- [10] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems*, Vol. 36.
- [11] Yuan Tian and Tianyi Zhang. 2026. PV-SQL: Synergizing Database Probing and Rule-Based Verification for Text-to-SQL Agents. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, San Diego, California, United States, 25827–25845. doi:10.18653/v1/2026.findings-acl.1286
- [12] Mattia Tritto, Giuseppe Farano, Dario Di Palma, Gaetano Rossiello, Fedelucio Narducci, Dharmashankar Subramanian, and Tommaso Di Noia. 2026. Test-Time Verification for Text-to-SQL via Outcome Reward Models. *arXiv preprint arXiv:2606.30851* (2026).
- [13] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics*. Association for Computational Linguistics, Abu Dhabi, UAE, 540–557.
- [14] Chi Wang, Qingyun Wu, Markus Weimer, and Erkang Zhu. 2021. FLAML: A Fast and Lightweight AutoML Library. In *Proceedings of Machine Learning and Systems*, Vol. 3. 434–447.
- [15] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir R. Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 3911–3921. doi:10.18653/v1/D18-1425

Supplementary Material

A FLAML Preprocessing Details

The main paper reports the FLAML training and model-selection configuration. For reproducibility, we additionally record the five input features that are not retained after FLAML preprocessing and feature transformation. All five belong to the Pair Analysis family. Their absence from the transformed representation reflects FLAML preprocessing rather than an explicit feature-selection procedure.

B Complete Feature Definitions

The unified representation contains 67 candidate-level features from five families: 10 ambiguity features, 5 question-planning features, 32 pair-analysis features, 10 deterministic SQL-structure features, and 10 intent-alignment features. Richer evidence—including explanations, rationales, confidence, warnings, and SQL fragments—is retained separately for traceability and is not included as additional model inputs. Canonical field names are shown in monospaced type where applicable; the main paper uses shortened display labels for readability.

B.1 Ambiguity Features

The ten ambiguity features are summarized in Table 8. Ambiguity is assessed once per question–database pair and shared across candidate SQL queries for the same question.

B.2 Question-Planning Features

The five question-planning features are summarized in Table 9.

B.3 Pair-Analysis Features

The 32 pair-analysis features are summarized in Tables 10 and 11. The model receives the canonical rule status for each feature. Rule scores and explanations are retained as diagnostic evidence but are not additional model features.

B.4 SQL-Structure Features

The candidate SQL is parsed using SQLGlot, and the resulting AST is inspected deterministically. The ten binary structure indicators are listed in Table 12. Parse failures remain diagnostic artifacts rather than being silently interpreted as a particular SQL structure.

Table 6: Pair Analysis input features not retained after FLAML preprocessing and feature transformation.

Feature
Versioning Rule Support
Name and Alias Support
Effective Date Meaning
User Context Mapping Support
User Context Resolution Support

Table 7: TraceSQL feature families.

Feature family	Count	Extraction
Ambiguity	10	Question-level ambiguity status, probability, and eight probe verdicts.
Question Planning	5	Probe verdicts comparing planned question requirements with candidate SQL support.
Pair Analysis	32	Direct projection of canonical Pair Analyzer rule statuses.
SQL Structure	10	Deterministic binary indicators extracted from the SQL AST.
Intent Alignment	10	Four evaluator fields and six probe-based alignment verdicts.
Total	67	Fixed candidate-level representation supplied to FLAML.

B.5 Intent-Alignment Features

Intent alignment combines four direct evaluator fields with six probe-based alignment verdicts. The ten resulting features are summarized in Table 13.

C Detailed Feature-Importance Analysis

The main paper reports the leading permutation-importance and SHAP results. Here we provide the global SHAP ranking and a cross-method comparison with permutation importance and native Extra Trees feature importance. These analyses characterize model reliance on the training/model-development population.

C.1 Global SHAP Analysis

Figure 2 reports mean absolute SHAP values for the fifteen highest-ranked features.

The global SHAP ranking is led by *Additional Grounding Check* (0.0199), followed by *SQL LIMIT Usage* (0.0147) and *Pattern-Based Business Meaning Support* (0.0117). *SQL DISTINCT Usage* (0.0074), *SQL Aggregation Usage* (0.0073), and *Data-Validity Rule Support* (0.0070) form the next group. The leading signals therefore include both deterministic SQL-structure properties and semantic grounding evidence.

A second group includes *SQL Grouping Usage* (0.0052), *Intent Alignment: Grouping* (0.0041), and *Question Planner: Output Structure* (0.0034). The remaining top-fifteen features have smaller mean absolute contributions.

C.2 Coverage Across Diagnostic Feature Families

The leading features are drawn from multiple parts of the TraceSQL representation rather than from a single diagnostic family. Table 15 summarizes the number of features from each family appearing in the top ten under SHAP, permutation importance, and native Extra Trees feature importance. This analysis describes representation coverage among highly ranked signals; it should not be interpreted as a feature-family ablation or as a measure of the causal contribution of each family.

Table 8: Ambiguity features used in TraceSQL.

Probe ID	Feature Name	Meaning
–	Ambiguity_Overall_Status	Overall Boolean assessment indicating whether the user question is considered ambiguous.
–	Ambiguity_Assessment_Probability	Estimated degree of ambiguity assigned by the Ambiguity Detector on a 0–100 scale.
AQ001	Ambiguity_Metric_Clarify	Captures uncertainty concerning the metric, measure, quantitative value, or derived calculation requested by the question.
AQ002	Ambiguity_Entity_Clarify	Captures uncertainty concerning the entity, subject, record, or target referenced by the question.
AQ003	Ambiguity_Time_Scope_Clarify	Captures uncertainty concerning the time period or temporal scope required by the question.
AQ004	Ambiguity_Constraint_Clarify	Captures uncertainty concerning filters, thresholds, limits, or other logical constraints expressed in the question.
AQ005	Ambiguity_Interpretation_Clarify	Captures whether the question admits multiple plausible semantic interpretations.
AQ006	Ambiguity_Reference_Resolution_Clarify	Captures unresolved references, antecedents, pronouns, or other implicit references in the question.
AQ007	Ambiguity_External_Context_Clarify	Captures whether interpretation of the question depends on unavailable business knowledge, domain conventions, or external context.
AQ008	Ambiguity_Analytical_Intent_Clarify	Captures uncertainty concerning the analytical objective or operation requested by the user.

Table 9: Question-planning features used in TraceSQL.

Probe ID	Feature Name	Meaning
QP001	Question_Planner_Analytical_Operation_Support	Whether the candidate SQL supports the analytical operation required by the question, such as lookup, aggregation, comparison, ranking, or trend analysis.
QP002	Question_Planner_Output_Structure_Support	Whether the candidate SQL supports the expected output columns, answer structure, and result granularity required by the question.
QP003	Question_Planner_Grouping_Support	Whether the candidate SQL supports the grouping requirements and aggregation dimensions required by the question.
QP004	Question_Planner_Ordering_or_Ranking_Support	Whether the candidate SQL supports ordering, ranking, top- <i>k</i> , sort direction, or extremum requirements expressed by the question.
QP005	Question_Planner_SQL_Operations_Support	Whether the candidate SQL contains the operations required to answer the question, such as joins, filters, aggregation, window functions, DISTINCT, or set operations.

Table 10: Pair-analysis features used in TraceSQL (Part I).

Rule ID	Feature Name	Meaning
schema_reference_verification	Pair_Analyzer_Schema_Reference_Support	Whether referenced schemas are present and supported by the catalog.
table_reference_verification	Pair_Analyzer_Table_Reference_Support	Whether referenced tables are present and supported by the catalog.
column_reference_verification	Pair_Analyzer_Column_Reference_Support	Whether referenced columns are present and supported by the catalog.
schema_scope_description_verification	Pair_Analyzer_Schema_Scope_Support	Whether schema-level descriptions support the question and SQL intent.
table_scope_description_verification	Pair_Analyzer_Table_Scope_Support	Whether table descriptions support the question and SQL intent.
column_scope_description_verification	Pair_Analyzer_Column_Scope_Support	Whether column descriptions support the question and SQL intent.
join_relationship_verification	Pair_Analyzer_Join_Relationship_Support	Whether the join relationship is explicitly supported by catalog metadata.
join_key_verification	Pair_Analyzer_Join_Key_Support	Whether the join keys are explicitly documented as relationship keys.
versioning_rule_verification	Pair_Analyzer_Versioning_Rule_Support	Whether required versioning, current-record, effective-dating, or snapshot rules are documented.
sql_identifier_alignment_verification	Pair_Analyzer_SQL_Name_Alignment	Whether SQL identifiers match catalog names or documented aliases.
naming_alias_verification	Pair_Analyzer_Name_and_Alias_Support	Whether abbreviations, prefixes, suffixes, or aliases used in SQL are documented.
identifier_semantics_verification	Pair_Analyzer_Identifier_Meaning_Support	Whether identifier columns used in joins or filters have documented business meaning.
numeric_operation_datatype_support_verification	Pair_Analyzer_Numeric_Data_Type_Support	Whether numeric operations use columns with compatible data types.
join_column_datatype_compatibility_verification	Pair_Analyzer_Join_Data_Type_Compatibility	Whether columns used in joins have compatible data types.
sql_datatype_support_verification	Pair_Analyzer_SQL_Data_Type_Support	Whether data-type metadata is sufficient to validate SQL behavior.
temporal_semantics_verification	Pair_Analyzer_Date_Time_Meaning_Support	Whether date/time columns have sufficient documented business meaning to validate how the SQL uses them.

Table 11: Pair-analysis features used in TraceSQL (Part II).

Rule ID	Feature Name	Meaning
temporal_fields_verification	Pair_Analyzer_Date_Time_Field_Distinction	Whether similar date/time fields are clearly distinguished so that the intended field can be identified.
temporal_queries_verification	Pair_Analyzer_Temporal_Analysis_Support	Whether temporal metadata supports historical, period-based, fiscal, or as-of analysis requested by the question.
timezone_semantics_verification	Pair_Analyzer_Timezone_Handling_Support	Whether timezone handling is documented sufficiently to interpret operational timestamps correctly.
effective_date_semantics_verification	Pair_Analyzer_Effective_Date_Meaning	Whether effective dates and validity periods clearly indicate when records become active, expire, or apply.
metric_definition_verification	Pair_Analyzer_Metric_Definition_Support	Whether a metric is documented sufficiently to validate the metric requested by the question.
metric_aggregation_logic_verification	Pair_Analyzer_Metric_Calculation_Support	Whether aggregations, ratios, and rollups follow the documented meaning of the metric.
table_role_semantics_verification	Pair_Analyzer_Table_Role_Clarity	Whether the catalog identifies the role of the table, such as master, transaction, history, snapshot, summary, or bridge.
null_semantics_verification	Pair_Analyzer_NULL_Handling_Support	Whether SQL handling of NULL values is consistent with documented NULL behavior.
data_validity_rule_verification	Pair_Analyzer_Data_Validity_Rule_Support	Whether predicates such as active, current, enabled, effective, latest, or approved are supported by documented business rules.
pattern_based_business_meaning_verification	Pair_Analyzer_Pattern_Based_Meaning_Support	Whether LIKE, regular-expression, substring, prefix, or encoded-code logic has documented business meaning.
pattern_mapping_verification	Pair_Analyzer_Pattern_Mapping_Support	Whether patterns used by the SQL map to documented business categories or values.
business_term_mapping_verification	Pair_Analyzer_Business_Term_Mapping_Support	Whether business terms in the question map to documented catalog entities, values, or terminology.
alias_or_canonical_mapping_verification	Pair_Analyzer_Canonical_Name_Mapping_Support	Whether aliases or alternate names map to documented canonical schema names or synonyms.
user_context_mapping_verification	Pair_Analyzer_User_Context_Mapping_Support	Whether user-context expressions such as “my”, “our”, or “assigned to me” map to documented business concepts.
user_context_resolution_verification	Pair_Analyzer_User_Context_Resolution_Support	Whether a user-context expression can be resolved to a documented catalog meaning or ownership relationship.
llm_detectable_pairwise_gap_verification	Pair_Analyzer_Additional_Grounding_Check	Whether any additional catalog or semantic grounding issue materially affects the reliability of the question and candidate SQL pair.

Table 12: Deterministic SQL-structure features used in TraceSQL.

Feature Name	Meaning
SQL Aggregation Usage	Whether the candidate contains an aggregate operation.
SQL Grouping Usage	Whether the candidate contains a GROUP BY operation.
SQL Join Usage	Whether the candidate joins multiple relations.
SQL Filtering Usage	Whether the candidate contains WHERE or HAVING filtering.
SQL Temporal Filter Usage	Whether the candidate contains a temporal/date-related filter.
SQL ORDER BY Usage	Whether the candidate contains an ORDER BY clause.
SQL LIMIT Usage	Whether the candidate contains a LIMIT clause.
SQL Subquery Usage	Whether the candidate contains a subquery or common table expression.
SQL Window Function Usage	Whether the candidate contains a window-function expression.
SQL DISTINCT Usage	Whether the candidate requests DISTINCT results.

Table 13: Intent-alignment features used in TraceSQL.

Feature Name	Meaning
Intent Alignment Overall Score	Reference-free evaluator score for overall alignment between the question and candidate SQL.
Intent Alignment Confidence	Confidence associated with the reference-free alignment evaluation.
Intent Alignment Gate Decision	Evaluator gate decision, such as accepting or requesting intervention.
Intent Alignment Intervention Reason	Categorized reason for intervention when the evaluator does not accept the candidate.
Intent Alignment: Requested Metric	Whether the metric or measure requested by the question is aligned with the candidate SQL.
Intent Alignment: Filter Conditions	Whether filtering constraints in the question are represented by the candidate SQL.
Intent Alignment: Entity or Scope	Whether the candidate SQL matches the requested entity and analytical scope.
Intent Alignment: Grouping	Whether grouping requirements in the question align with the candidate SQL.
Intent Alignment: Ranking or Extremum	Whether ranking, top- <i>k</i> , minimum, maximum, or related requirements align with the candidate SQL.
Intent Alignment: Business-Term Mapping	Whether business terminology in the question is aligned with the schema and SQL interpretation.

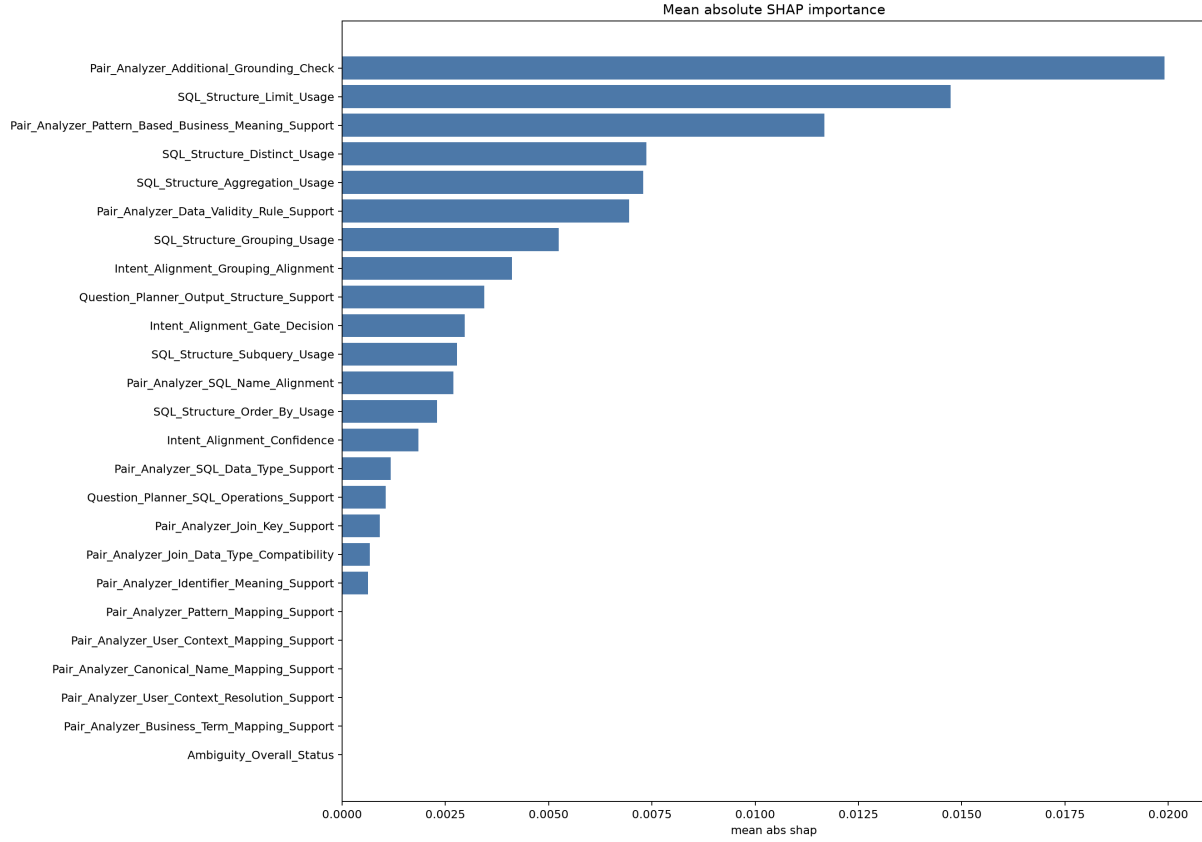


Figure 2: Mean absolute SHAP importance for the fifteen most influential TraceSQL features. Larger values indicate greater average contribution to the fitted model output.

Table 14: Cross-method agreement among the leading TraceSQL features. Lower rank indicates greater importance within each method.

Feature	SHAP	Perm.	Native
Additional Grounding Check	1	4	2
SQL LIMIT Usage	2	2	5
Pattern-Based Business Meaning Support	3	5	3
SQL DISTINCT Usage	4	1	14
SQL Aggregation Usage	5	3	1
Data-Validity Rule Support	6	6	4
SQL Grouping Usage	7	9	8
Intent Alignment: Grouping	8	7	17
Question Planner: Output Structure	9	8	12
SQL Subquery Usage	11	10	16

Across all three methods, SQL Structure and Pair Analysis account for most of the highly ranked features, while Question Planning and Intent Alignment also contribute signals to the leading set. Importantly, the rankings do not reduce to SQL structure alone: semantic grounding and validity signals from the Pair Analysis family consistently appear among the strongest features.

Table 15: Feature-family representation among the top ten features under each importance method.

Feature Family	SHAP	Perm.	Native
Ambiguity	0	0	0
Question Planning	1	1	1
Pair Analysis	3	3	4
SQL Structure	4	5	3
Intent Alignment	2	1	2

C.3 Cross-Method Feature-Importance Comparison

Permutation importance, SHAP, and native Extra Trees feature importance measure different aspects of model reliance. Table 14 therefore compares ranks rather than placing their raw values on a common scale.

The strongest cross-method agreement is observed for *Additional Grounding Check*, *SQL LIMIT Usage*, *Pattern-Based Business Meaning Support*, *SQL Aggregation Usage*, and *Data-Validity Rule Support*, all of which rank near the top under SHAP, permutation importance, and native feature importance. *SQL DISTINCT Usage* is more method-sensitive: it ranks first under permutation importance

and fourth under SHAP, but lower under native feature importance. Overall, the recurring high-ranked set spans both SQL-structure and semantic diagnostic signals. Ambiguity features do not appear within the top ten under the three importance methods. In the current training, ambiguity is assessed at the question–database level and is therefore shared across alternative candidate SQLs associated with the same question. In contrast, SQL-structure, grounding, and intent-alignment features can vary across candidates. This difference may reduce the ability of ambiguity features to distinguish positive and negative candidate SQLs within the paired training setting. The result should therefore be interpreted as limited model reliance on the current ambiguity representation under this training distribution rather than as evidence that question ambiguity is uninformative for Text-to-SQL verification.

D Feature Correlation Analysis

We examine pairwise correlations among numeric TraceSQL features to identify strongly related signals that may contain overlapping information.

Most numeric feature pairs exhibit weak linear relationships. Two notable dependencies are *SQL ORDER BY Usage* and *SQL LIMIT Usage* ($r = 0.887$), and *Intent Alignment Overall Score* and *Intent Alignment Confidence* ($r = 0.729$). These dependencies provide context for differences across feature-importance methods, since correlated features can share or redistribute predictive information. The correlations are descriptive of the training population.

E Representative Prediction Trace

To illustrate candidate-level traceability, we examine a representative false-positive case from the generated-SQL evaluation. The question asks for the top three drivers and their points in the 2017 Chinese Grand Prix. The generated candidate SQL filters the relevant race, joins the race, results, and driver tables, orders the candidates by points, and applies `LIMIT 3`. The execution-match label is negative, while TraceSQL assigns a positive probability of 0.572.

The retained diagnostic evidence makes the source of this prediction inspectable. The candidate SQL contains structural signals that are globally influential in the fitted model, including ordering, `LIMIT`, filtering, and joins. Pair Analyzer diagnostics support the referenced tables, columns, and join relationship, while the intent-alignment diagnostics mark ranking and business-term alignment as *PASS*, with an overall intent score of 1.0 and an *ACCEPT* gate decision. Multiple diagnostic signals therefore describe the candidate as plausible even though its execution-match label is negative.

This case illustrates the feature-to-evidence path used for traceability: local attribution identifies influential model features, and those features remain linked to named SQL-structure properties, Pair Analyzer rule results, or probe-level diagnostic records retained during feature extraction.

F Additional Database-Level Results

The main paper reports the aggregate ground-truth SQL evaluation. Because all 1,534 instances belong to the positive class, accuracy equals recall, precision is 100%, and ROC-AUC is undefined. We therefore report database-level acceptance rates in Table 16.

Table 16: Database-level acceptance rate (%) on the 1,534 ground-truth SQL queries. Because all instances belong to the positive class, acceptance rate is equivalent to accuracy and recall. Bold indicates the higher result.

Database	GradeSQL-7B	TraceSQL
California Schools	41.57	67.42
Card Games	59.16	43.98
Codebase Community	65.59	66.13
Debit Card Specializing	68.75	29.69
European Football 2	65.12	62.79
Financial	60.38	75.47
Formula 1	65.52	72.41
Student Club	73.42	60.76
Superhero	79.84	65.12
Thrombosis Prediction	50.92	54.60
Toxicology	57.93	55.86
Overall	62.84	60.17

TraceSQL has higher acceptance on California Schools, Codebase Community, Financial, Formula 1, and Thrombosis Prediction, while GradeSQL-7B is higher on the remaining six databases. The generated-SQL setting remains the primary two-class verification evaluation.

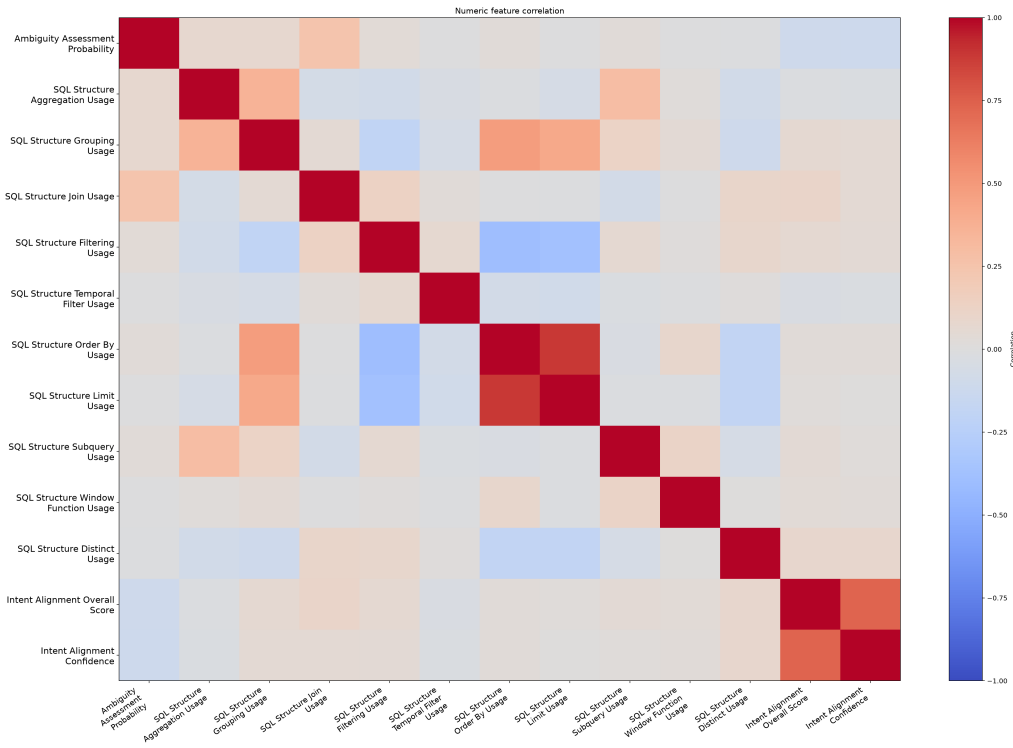


Figure 3: Pairwise correlation matrix for the numeric TraceSQL features.

Local explanation for a failed prediction — actual: 0, predicted: 1

Feature and observed value

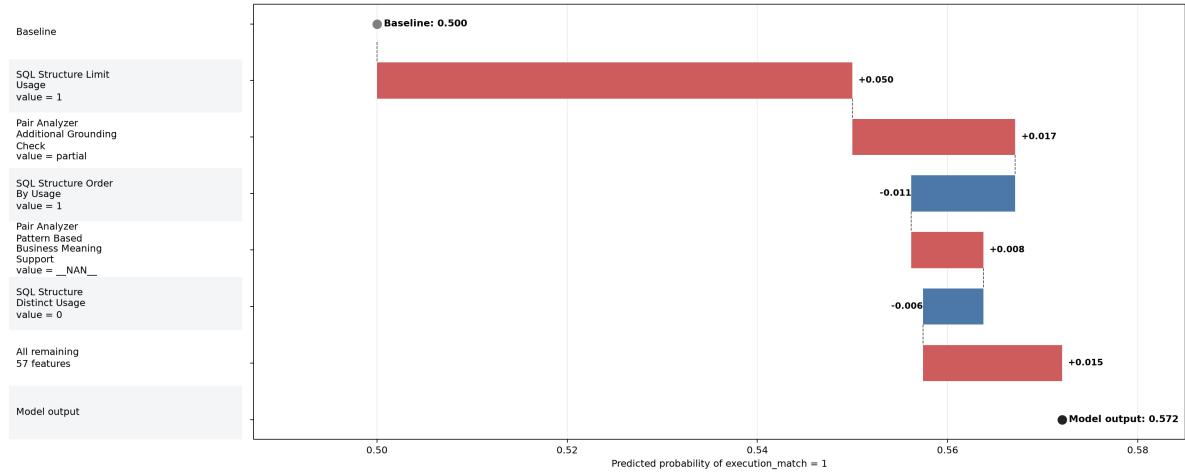


Figure 4: Local SHAP explanation for the representative false-positive verification decision.